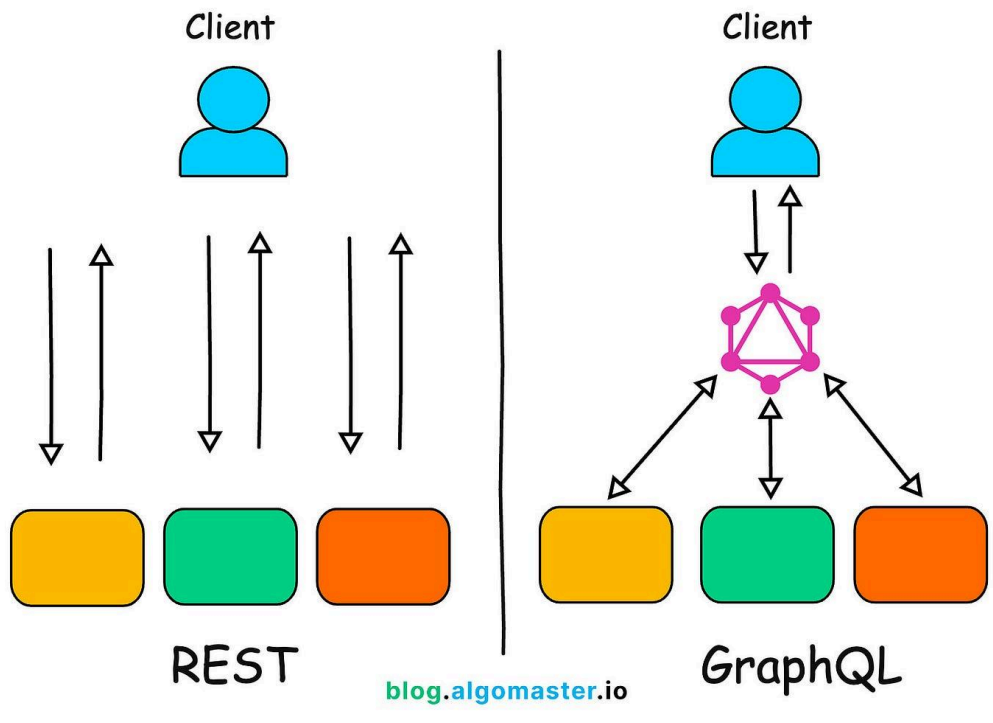
3.GraphQL

What is GraphQL ?



- GraphQL is a query language for APIs and a runtime for executing those queries
- Created by Facebook in 2012, open-sourced in 2015
- It is not a database query language — it has nothing to do with SQL or databases directly
- It is a specification for how clients and servers communicate
- The client describes exactly what data it needs, the server returns exactly that — nothing more, nothing less
- One endpoint handles everything — unlike REST which has one URL per resource, GraphQL has one URL for the entire API, typically POST /graphql

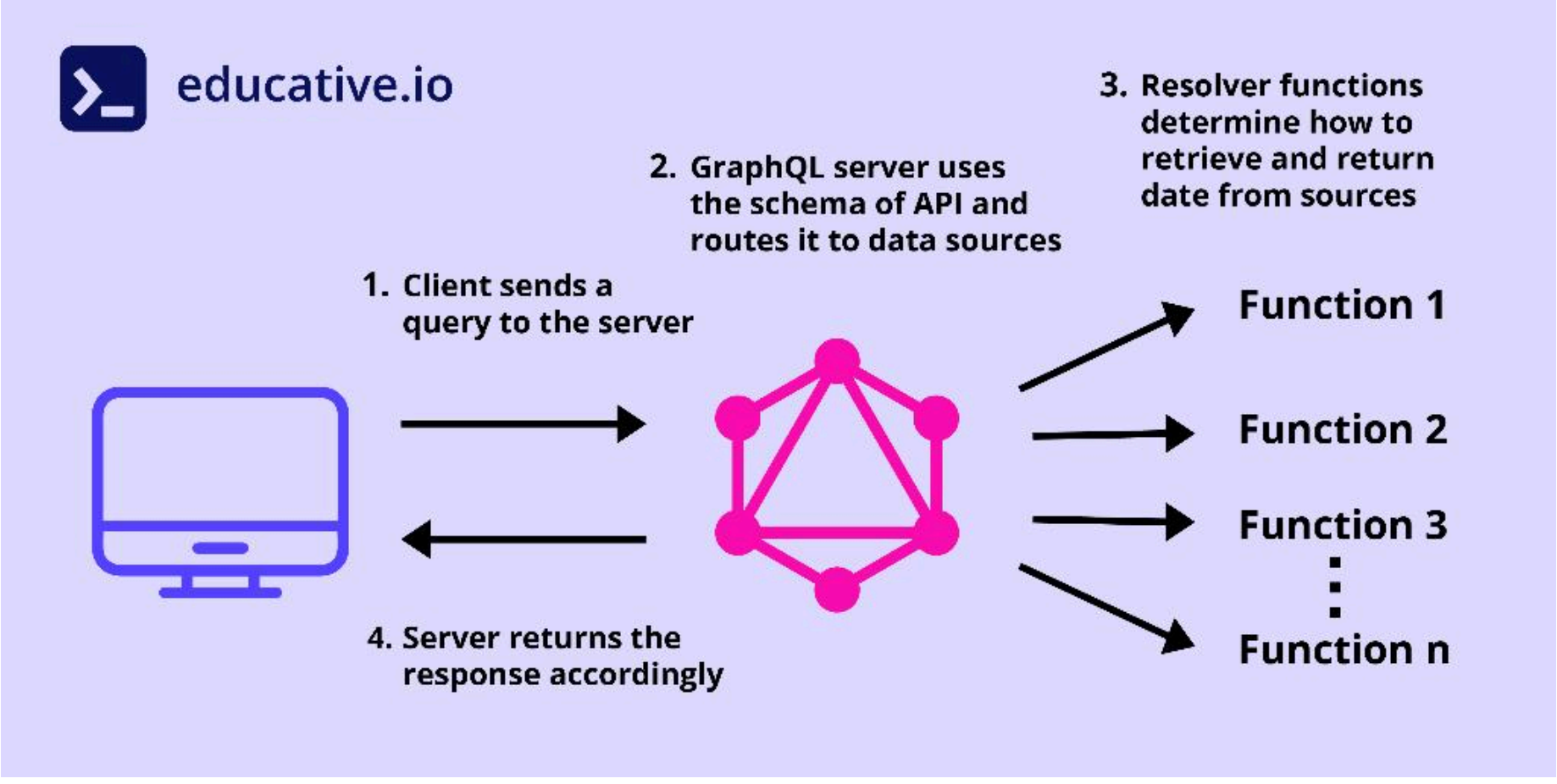
---

Why GraphQL Was Created

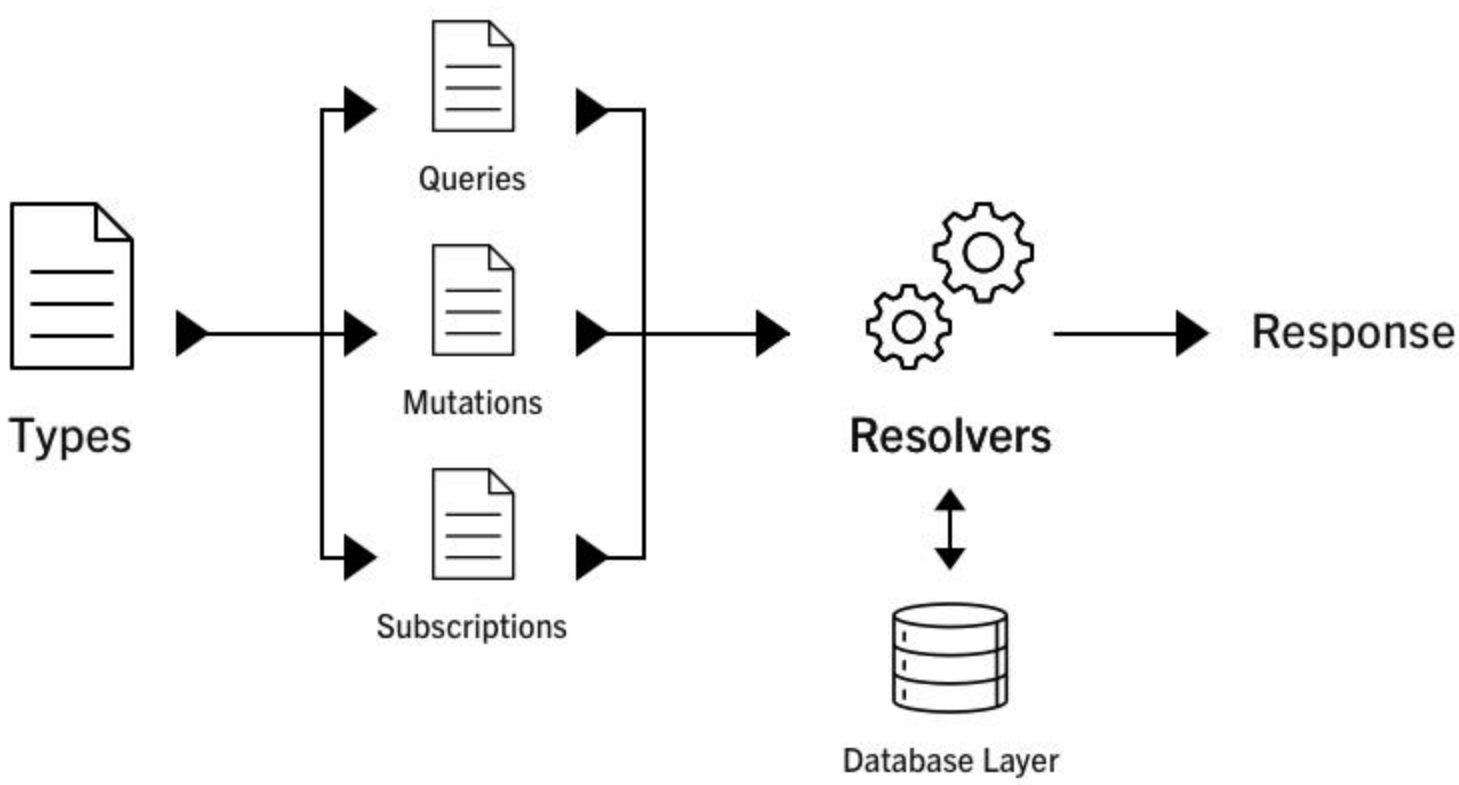
- Facebook built it to solve real problems they hit while building the Facebook mobile app in 2012
- The problems were direct consequences of REST's limitations at scale
- **Over-fetching** — REST returns entire resource objects, mobile clients only needed specific fields, wasting bandwidth on slow mobile networks
- **Under-fetching** — a single screen needed data from multiple REST endpoints, causing multiple round trips, each adding latency on mobile
- **Rapid product iteration** — Facebook's product teams were constantly changing what data UI screens needed, every change required backend API changes and new endpoints
- **Versioning pain** — REST versioning was creating a proliferation of endpoints and versions that were hard to maintain
- **Strongly typed contracts** — REST had no formal schema, field changes broke clients silently
- GraphQL solved all of these in one shot — client declares exactly what it needs, server returns exactly that, one round trip, one endpoint, strongly typed schema

---

How GraphQL Works



- Client sends a POST request to a single endpoint (typically /graphql)
- The request body contains a GraphQL query — a string describing exactly what data is needed
- The GraphQL server parses the query, validates it against the schema, executes it, and returns exactly the requested fields
- Response is always JSON
- Response always has a data key containing the result and optionally an errors key
- The flow:
  - Client writes a query describing needed data
  - Query is sent as POST body to /graphql
  - Server validates query against schema — if invalid, returns error immediately
  - Server resolves each field using resolver functions
  - Server assembles the response matching the exact shape of the query
  - Client receives data in exactly the shape it asked for



Schema Definition

- The schema is the heart of GraphQL
- It is a strongly typed contract between client and server
- Defines every type, every field, every operation available in the API
- Written in SDL — Schema Definition Language
- Both client and server agree on the schema — client can only query what the schema defines

- If a client requests a field that does not exist in the schema, GraphQL rejects the query before execution

Types

- Everything in GraphQL is a type
- Types define the shape of your data
- There are two categories — built-in scalar types and custom object types
- Object type definition example:

```
type User {
  id: ID!
  name: String!
  email: String!
  age: Int
  isActive: Boolean!
  orders: [Order!]!
}

type Order {
  id: ID!
  total: Float!
  status: String!
  createdAt: String!
  user: User!
}
```

- The ! means non-nullable — this field will never be null
- Without ! the field is nullable — it may return null
- [Order!]! means a non-nullable list of non-nullable Orders

Fields

- Fields are the properties on a type
- Every field resolves to either a scalar value or another type
- Fields can take arguments:

```
type User {
  id: ID!
  name: String!
  orders(status: String, limit: Int): [Order!]!
}
```

- Here orders is a field that accepts filter arguments

Scalars

- Scalars are the primitive leaf types — they resolve to a concrete value
- Built-in scalars:
  - String — UTF-8 text
  - Int — 32-bit integer
  - Float — double precision floating point
  - Boolean — true or false
  - ID — unique identifier, serialised as a string
- Custom scalars can be defined for specific types:
  - Date, DateTime, Email, URL, JSON
  - Example: scalar DateTime
  - You define how it is serialised and deserialised

Queries

How queries work

- A query is a read operation — equivalent to GET in REST
- Client writes a query that mirrors the shape of the data it wants

- The response JSON matches the exact shape of the query
- The schema defines a special Query type that lists all available read operations
- Schema side:

```
type Query {
  user(id: ID!): User
  users(limit: Int, offset: Int): [User!]!
  product(id: ID!): Product
}
```

- Client query:

```
query {
  user(id: "123") {
    id
    name
    email
  }
}
```

- Response:

```
{
  "data": {
    "user": {
      "id": "123",
      "name": "Arjun",
      "email": "arjun@example.com"
    }
  }
}
```

- The server fetched the full user object but only returned the three fields the client asked for
- age, isActive, orders — none of these were requested so none were returned

Nested queries

- This is where GraphQL's power really shows
- You can fetch related data in a single query by nesting fields
- In REST this would require multiple round trips

```
query {
  user(id: "123") {
    id
    name
    orders {
      id
      total
      status
      items {
        productName
        quantity
        price
      }
    }
  }
}
```

- One query, one round trip — fetches user, their orders, and each order's items
- In REST: GET /users/123, then GET /users/123/orders, then GET /orders/{id}/items for each order
- GraphQL collapses all of this into a single request

Query variables

- Hardcoding values inside queries is bad practice

- Query variables let you parameterise queries — separate the query structure from the dynamic values
- Query with variable:

```
query GetUser($userId: ID!) {
  user(id: $userId) {
    id
    name
    email
  }
}
```

- Variables sent separately in the request:

```
{
  "userId": "123"
}
```

- Why this matters:
  - Prevents injection attacks — variables are typed and sanitised separately
  - Queries can be cached on the server since the query string itself never changes
  - Cleaner, reusable query definitions

Mutations

How mutations work

- Mutations are write operations — create, update, delete
- Equivalent to POST, PUT, PATCH, DELETE in REST
- Schema defines a Mutation type listing all available write operations
- Mutations can return data — you can fetch updated state in the same operation
- Schema side:

```
type Mutation {
  createUser(input: CreateUserInput!): User!
  updateUser(id: ID!, input: UpdateUserInput!): User!
  deleteUser(id: ID!): Boolean!
}

input CreateUserInput {
  name: String!
  email: String!
  age: Int
}
```

- Note the input type — a special type used for mutation arguments, keeps arguments organised

Create via mutation

```
mutation {
  createUser(input: {
    name: "Arjun"
    email: "arjun@example.com"
    age: 25
  }) {
    id
    name
    email
  }
}
```

- Creates the user and immediately returns the created user's id, name, email in one operation
- In REST you POST to create, then GET to fetch the created resource — two calls
- GraphQL does it in one

Update via mutation

```
mutation UpdateUser($id: ID!, $input: UpdateUserInput!) {
  updateUser(id: $id, input: $input) {
    id
    name
    email
    updatedAt
  }
}
```

- Updates the user and returns the updated fields immediately

Delete via mutation

```
mutation {
  deleteUser(id: "123")
}
```

- Returns true if deleted, false if not found
- Or you can return the deleted object for confirmation

Subscriptions

How subscriptions work

- Subscriptions are real-time operations — the server pushes updates to the client
- Unlike queries and mutations which are request-response, subscriptions are long-lived connections
- Client subscribes to an event, server sends data whenever that event occurs
- Implemented over WebSockets — a persistent bidirectional connection between client and server
- Schema defines a Subscription type:

```
type Subscription {
  orderStatusUpdated(orderId: ID!): Order!
  newMessage(conversationId: ID!): Message!
  userTyping(conversationId: ID!): User!
}
```

The flow

- Client sends a subscription query over WebSocket
- Server registers the subscription for that client
- When the specified event occurs (order status changes, new message arrives), server pushes the update to all subscribed clients
- Connection stays open until client unsubscribes or disconnects

Real-time data via subscriptions

```
subscription {
  orderStatusUpdated(orderId: "789") {
    id
    status
    updatedAt
    estimatedDelivery
  }
}
```

- Client opens a WebSocket, sends this subscription
- Whenever order 789's status changes, server pushes the updated order data to this client
- Client does not need to poll — server pushes proactively

Use cases

- Live chat — new messages pushed to all participants

- Live notifications — new order, payment confirmed, delivery update
- Collaborative editing — changes pushed to all users editing the same document
- Live dashboards — metrics updated in real time
- Sports scores, stock prices — live data feeds

N+1 Problem

What it is

- The N+1 problem is a performance issue where fetching a list of N items results in N+1 database queries
- 1 query to fetch the list, then N separate queries to fetch related data for each item
- Example: fetch 10 users → 1 query. Fetch each user's orders → 10 separate queries. Total: 11 queries.
- As N grows, database load grows linearly — catastrophic at scale

Why it happens in GraphQL

- GraphQL resolves each field independently using resolver functions
- A resolver is a function that fetches data for a specific field
- When you query:

```
query {
  users {
    id
    name
    orders {
      id
      total
    }
  }
}
```

- GraphQL executes:
  - users resolver → SELECT * FROM users → returns 100 users
  - For each user, GraphQL calls the orders resolver with that user's ID
  - orders resolver for user 1 → SELECT * FROM orders WHERE user_id = 1
  - orders resolver for user 2 → SELECT * FROM orders WHERE user_id = 2
  - ... repeated 100 times
  - Total: 1 + 100 = 101 database queries
- Each resolver works in isolation — it does not know about the other resolver calls happening in parallel
- This is the N+1 problem in GraphQL

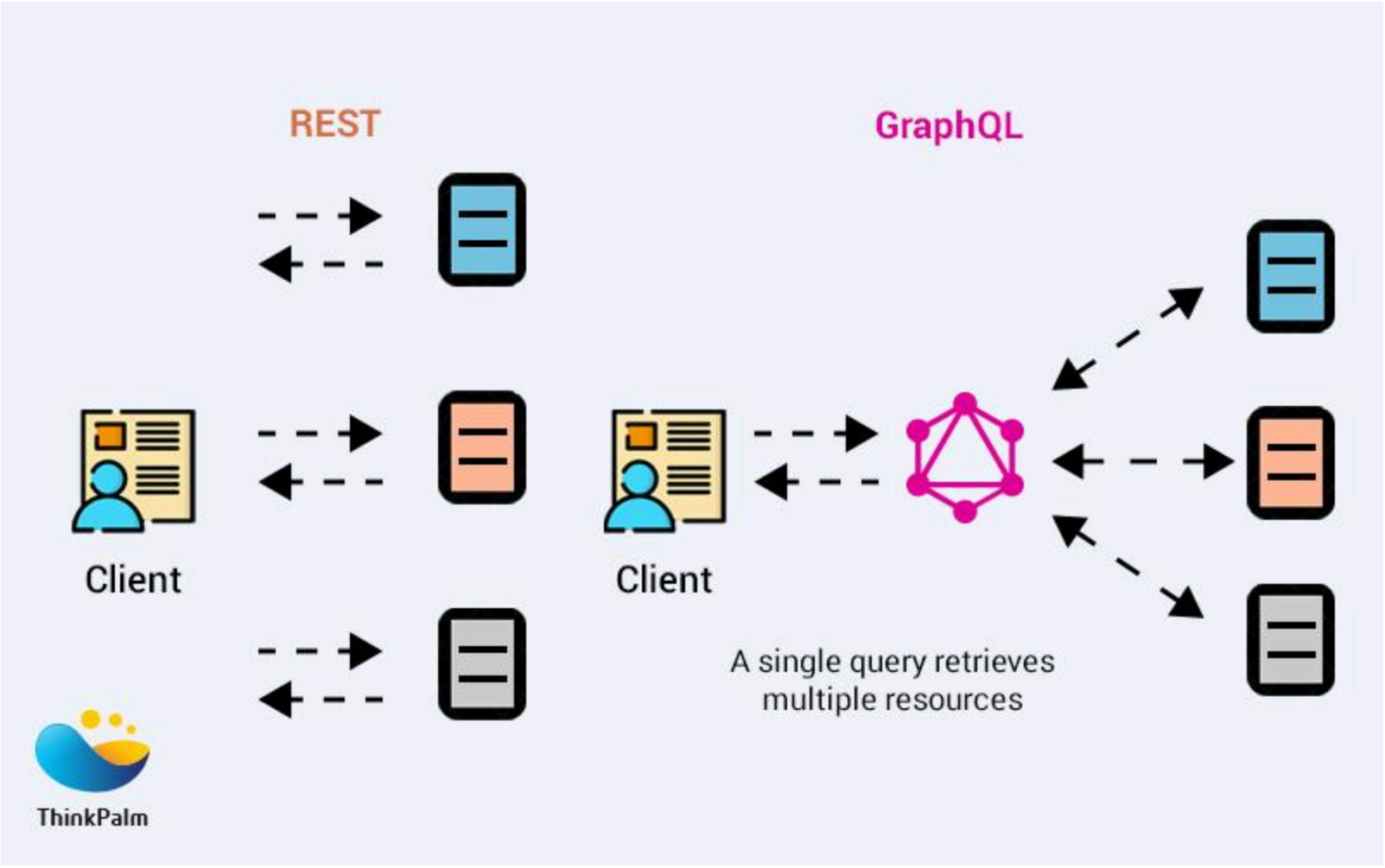
DataLoader as the solution

- DataLoader is a utility developed by Facebook to solve N+1 in GraphQL
- It works using two techniques — batching and caching
- **Batching**
  - Instead of firing a database query for each individual user ID, DataLoader collects all the IDs being requested in the current tick of the event loop
  - After the current tick, it fires one batched query: SELECT * FROM orders WHERE user_id IN (1, 2, 3, ... 100)
  - Returns results mapped back to each requesting resolver
  - 101 queries collapsed into 2 queries
- **Caching**
  - Within a single request, if the same ID is requested multiple times, DataLoader returns the cached result
  - Prevents duplicate queries within the same request lifecycle
- **The flow with DataLoader**
  - users resolver runs, returns 100 users
  - GraphQL starts resolving orders for each user

    - Each orders resolver calls dataloader.load(userId) instead of querying directly
    - DataLoader collects all 100 userIds during the current event loop tick
    - After the tick, DataLoader fires: SELECT * FROM orders WHERE user_id IN (1..100)
    - Maps results back to each resolver
    - 101 queries → 2 queries
- DataLoader is per-request — a new instance is created per GraphQL request to avoid cross-request data leaking

---

When to Use GraphQL vs REST



Use GraphQL when

- Multiple clients with different data needs — mobile app needs less data than web app, GraphQL lets each client ask for exactly what it needs
- Complex, interconnected data — social graphs, product catalogues with relationships, content platforms
- Rapid frontend iteration — product teams change data requirements frequently, no need to create new backend endpoints
- Reducing over-fetching and under-fetching is critical — bandwidth-constrained environments like mobile
- Real-time features needed alongside regular queries — subscriptions handle this natively
- Strong typing and schema introspection are valuable — auto-generated documentation, type-safe client code generation

Use REST when

- Simple CRUD API with straightforward resource model
- Public API for third-party developers — REST is universally understood, easier to onboard developers
- File uploads and downloads — REST handles these naturally, GraphQL does not
- Heavy HTTP caching is required — REST's URL-per-resource model enables granular caching, GraphQL's single endpoint makes HTTP caching very difficult
- Team is small and familiar with REST — GraphQL has a steeper learning curve and more infrastructure complexity
- Microservice-to-microservice communication — REST or gRPC is more appropriate than GraphQL

The honest answer for system design interviews

- Default to REST for most designs
- Reach for GraphQL when you explicitly have the problems it solves — multiple clients, complex data shapes, frequent UI changes

- Mention GraphQL as a consideration when designing systems like social networks, e-commerce with rich product data, or content platforms

---

GraphQL Limitations

Caching complexity

- REST's URL-per-resource model maps perfectly to HTTP caching — each URL is a cache key
- GraphQL uses a single POST endpoint for all queries — HTTP caches cannot differentiate between different queries
- Browser cache, CDN cache, reverse proxy cache — all work on URLs, all are useless for GraphQL out of the box
- Solutions exist but add complexity:
  - Persisted queries — client pre-registers queries, server assigns each a hash, client sends GET request with hash as URL param — now it is cacheable
  - Apollo Client has its own client-side cache with normalised entity caching
  - But these are application-level solutions, not HTTP-level caching
- This is a serious limitation for public APIs and content with high read traffic

Over-fetching queries — the client-side problem

- GraphQL prevents server-side over-fetching but cannot prevent client-side over-fetching
- A developer can write a lazy query that requests every field on every type
- Nothing in the protocol stops a client from requesting all fields — equivalent to REST over-fetching
- Discipline and code review are the only guard against this
- Query complexity analysis tools can limit excessively greedy queries

Security concerns

- **Query depth attacks**
  - GraphQL allows deeply nested queries
  - A malicious client could send:

```
{ user { friends { friends { friends { friends { friends { ... } } } } } } }
```

  - Each level multiplies the database load
  - 10 levels of nested friends on a social graph could result in billions of records being fetched
  - Solution: implement query depth limiting — reject queries beyond a certain nesting depth
- **Query complexity attacks**
  - Even without deep nesting, a query requesting many expensive fields can be a DoS attack
  - Solution: assign a cost to each field, reject queries exceeding a total cost threshold
- **Introspection in production**
  - GraphQL's schema is self-documenting — clients can query the schema itself using introspection
  - This is extremely useful in development
  - In production, introspection exposes your entire API structure to attackers
  - Best practice: disable introspection in production
- **Batched query attacks**
  - GraphQL supports sending multiple queries in one request
  - Attacker sends 1000 mutations in one request — effectively a batch attack
  - Solution: limit batch size and implement rate limiting at the query level, not just the HTTP level
- **Authorization complexity**
  - In REST, authorization is per endpoint — easy to reason about
  - In GraphQL, the same type can be accessed from multiple queries and mutations
  - Authorization must be implemented at the resolver level — every resolver must check permissions
  - Easy to accidentally leave a resolver unprotected

- Solution: use a middleware layer or directive-based authorisation to enforce it consistently